

Ecosystem Simulation — Program Design

COMP2000 · Worksheet 7 supporting document · Arjun Sandeep Rao Sahab

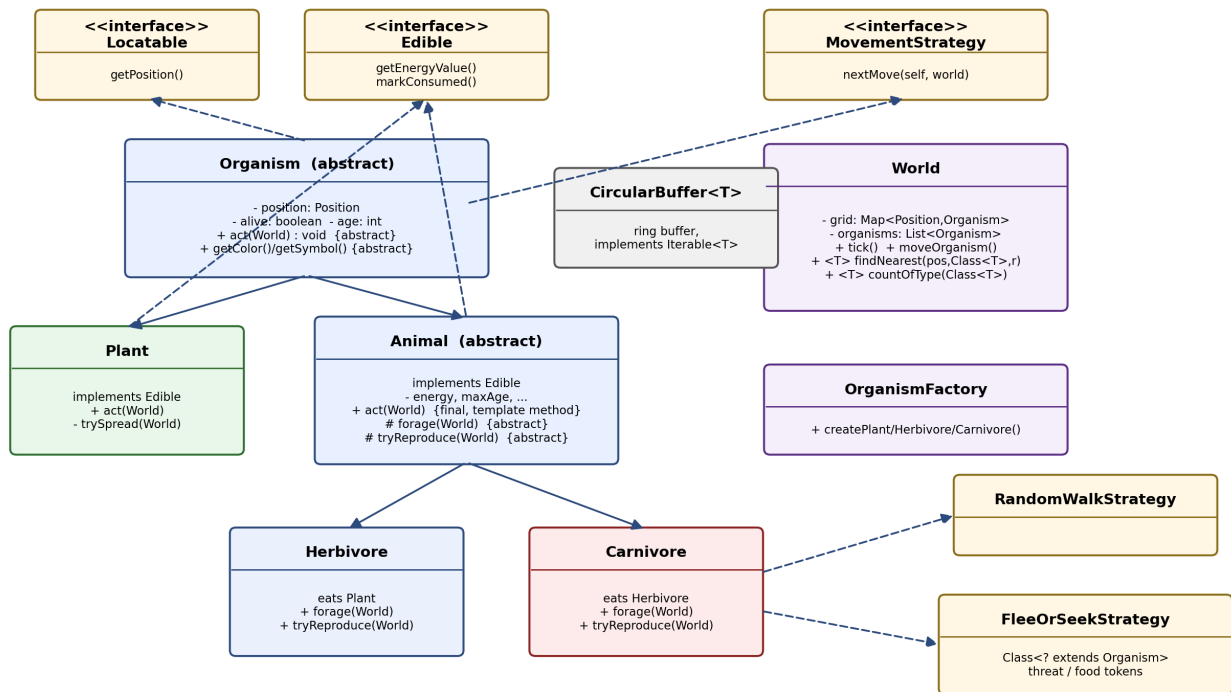
This document is the design write-up referenced by worksheet-7.md. It covers what the simulation is, the class hierarchy and why it is shaped the way it is, where generics and exceptions are used and why, and the two design patterns used to keep species-specific behaviour out of shared code. The simulation itself is a predator–prey–plant ecosystem on a 60×40 grid: plants grow and spread, herbivores eat plants and flee carnivores, and carnivores hunt herbivores. All three populations rise and fall in the classic boom/bust cycle this kind of agent-based model is known for, which is the emergent behaviour the assignment asks for.

1. Class overview

Class / Interface	Responsibility
Position	Immutable (x, y) grid coordinate. All movement goes through World, never by mutating a Position.
Locatable / Edible	Interfaces: "has a position" and "can be eaten" respectively.
Organism (abstract)	Shared state for anything on the grid: position, alive flag, age. Declares act(World) and the rendering hooks getColor()/getSymbol() as abstract.
Plant	Stationary Edible. Grows with age and, once mature, may spread a copy of itself into a free neighbouring cell.
Animal (abstract)	Shared state/behaviour for things that move and eat: energy, upkeep, ageing, starvation. act() is a final template method; forage() and tryReproduce() are the abstract hooks subclasses fill in.
Herbivore / Carnivore	Leaf classes. Herbivore eats Plant and flees Carnivore; Carnivore eats Herbivore and has no predator.
MovementStrategy (+ RandomWalkStrategy, FleeOrSeekStrategy)	Strategy pattern: decides where an Animal moves this tick. See section 3.
OrganismFactory	Simple Factory: the one place that knows how to build a fresh Plant/Herbivore/Carnivore, including its starting energy and movement strategy.
World	Owns the grid (Map<Position,Organism>) and the master list of organisms. Runs tick(), enforces move/placement legality, and exposes the generic search/count helpers used throughout.
CircularBuffer<T>	A small generic ring buffer (implements Iterable<T>) used to keep bounded-length population history for the on-screen sparkline.
SimulationException / InvalidPositionException / OverpopulationException	Custom checked exception hierarchy. See section 4.
WorldPanel / SimulationFrame / SmokeTest / Main	Swing GUI (JPanel + JFrame, standard JRE only), a headless console test harness, and the entry point.

2. Class diagram

Solid arrows are inheritance (extends); dashed arrows are interface implementation, or, for CircularBuffer and OrganismFactory, a "depends on / uses" relationship.



solid arrow = inheritance (extends) dashed arrow = interface implementation / uses

Ecosystem Simulation - class relationships (ecosim package)

3. Inheritance hierarchy and design pattern rationale

Organism → Plant, Organism → Animal → {Herbivore, Carnivore}. Organism captures what every single thing on the grid needs regardless of species (a position, an alive flag, an age, and the requirement that it can act() once per tick and render itself). Animal sits between Organism and the two leaf species because Herbivore and Carnivore share a large amount of behaviour that Plant does not need at all: energy, upkeep cost, starvation, movement, and reproduction thresholds. Putting that in Animal instead of duplicating it in both leaf classes (or, worse, hoisting it into Organism where Plant would inherit fields it never uses) is the whole justification for the extra layer.

Template Method (Animal.act). Animal.act(World) is marked final and defines the fixed sequence every animal follows each tick: age, pay upkeep, check for starvation/old age, move, forage, maybe reproduce. Herbivore and Carnivore cannot change that order — they can only plug into it via the two abstract hooks forage(World) and tryReproduce(World). This was chosen over letting each subclass implement its own act() because the ageing/starvation bookkeeping is exactly the kind of logic that quietly drifts out of sync between two hand-written copies; writing it once in the base class and sealing it with final removes that risk entirely.

Strategy (MovementStrategy). How an animal decides where to move is independent of the template method above, and needed its own abstraction because "flee my predator, otherwise seek my food, otherwise wander" is genuinely reusable logic — Herbivore and Carnivore just plug in different threat/food types. RandomWalkStrategy and FleeOrSeekStrategy implement MovementStrategy, and OrganismFactory wires the right one to each species at construction time, so Animal itself never has to know which concrete strategy it is holding.

Simple Factory (OrganismFactory). Both "a herbivore reproduces" and "the world is seeded at startup" need to create a brand-new, fully-configured organism. Centralising that in one factory class means the starting energy, reproduction constants and movement strategy for a species are defined in exactly one place rather than copy-pasted at every call site.

4. Generics

Off-the-shelf generics are used throughout (List<Organism>, Map<Position,Organism>, Optional<T>), but two uses were designed deliberately rather than reached for by default:

- **World.findNearest / World.countOfType** — generic methods parameterised with a Class<T> token. Before this, "find the nearest Plant" and "find the nearest Carnivore" would have been two near-identical methods (or one method returning Organism with a manual cast at every call site, which is exactly what generics exist to avoid). <T extends Organism> Optional<T> findNearest(Position, Class<T>, int) lets FleeOrSeekStrategy ask for a specific species back, fully typed, from one shared implementation.
- **CircularBuffer<T>** — a small generic type written for this project (not from the JDK) so the same bounded ring-buffer logic serves the three per-species population histories without being hard-coded to int or Integer.

Where generics were deliberately not used: World itself is not generic (not World<T extends Organism>), even though it looks like an obvious candidate. World manages a heterogeneous mix of Plant, Herbivore and Carnivore at the same time, so a single type parameter on the class would not describe what it actually stores — the targeted, method-level generics above give type safety exactly where it is needed (a specific species) without forcing an incorrect single-type constraint onto the whole class.

5. Exceptions

`SimulationException` is an abstract checked base class with two subclasses: `InvalidPositionException` (a move or placement target is out of bounds or occupied) and `OverpopulationException` (the grid is full). Both are checked, not unchecked, on purpose: a crowded grid is an expected, recoverable condition in an ecosystem simulation, not a programming error, so every caller is forced by the compiler to decide how to recover rather than letting a bad tick silently propagate and crash the Swing event thread.

The same `OverpopulationException` is deliberately handled three different ways in three different places, which is worth calling out explicitly because it is the clearest evidence of exceptions being integrated thoughtfully rather than bolted on:

- **`Plant.trySpread / Herbivore.tryReproduce / Carnivore.tryReproduce`** — caught immediately and silently ignored. A full grid just means "no offspring this tick", which happens routinely and is not worth logging.
- **`World.tick()`** — caught centrally around each organism's `act()` call and tallied into an `overpopulationEvents` counter (shown live in the GUI), so one crowded organism cannot abort the whole tick, but the aggregate frequency is still visible.
- **`SimulationFrame.seedWorld()`** — allowed to propagate out of the seeding loop to a top-level catch that shows a `JOptionPane` dialog. Too many initial organisms for the chosen grid size is a configuration mistake, not routine crowding, so here the user is told immediately instead of the failure being swallowed.

`InvalidPositionException` is handled at a single, narrow point — `Animal.move()` — where it is caught and treated as "stay where you are this tick". It is never declared on `act()` or allowed to propagate further, because by the time `World.moveOrganism` throws it, the only sensible recovery (try a different cell, or wait) is entirely local to the animal that was trying to move.

6. Encapsulation and polymorphism

`Position` is immutable — every field is final and `translate()` returns a new instance — so obtaining an organism's position via `getPosition()` can never be used to move it; the only legal way to change an organism's location is `World.moveOrganism`, which is exactly where bounds/collision rules live. `Organism.setPosition` is package-private for the same reason: only `World` (same package) is allowed to relocate an organism.

Polymorphism shows up most visibly in `WorldPanel.paintComponent`, which draws every organism by calling `organism.getColor()` and checking only instanceof `Plant` to choose a square-vs-circle shape — there is no instanceof chain to distinguish `Herbivore` from `Carnivore`. Each species overrides `getColor()` to say how it should look, so adding a fourth species later would need no changes to the rendering code at all.

7. How to run it

GUI: compile everything under `src/` and run `ecosim.Main` (`javac -d bin $(find src -name "*.java") && java -cp bin ecosim.Main`). Only standard JRE classes are used — no external libraries or build tool are required. Headless check: `java -cp bin ecosim.SmokeTest` runs 400 ticks with no GUI and prints population counts every 20 ticks, useful for confirming the model logic without watching the window.